コンパイラ再評価者向け演習: cit-compiler-special

> 立合もしくは監督の方は、演習実施前に再評価演習の受講者へ心得の指示をお願いします。
>
> 受講生は教員の指示に従うこと。

1. 演習の概要

これは、コンパイラの再評価者向けの演習のAssignmentです。

1. Lox言語でプログラムを作成して動かす。
2. 自分のLoxインタプリタを動作させる。

2. 演習の準備

> 基本的には講義中の演習を実施していたのであれば、これらの準部はすでに済んでいるはず。
> もし、講義中に準備していなかったものがあれば、最初にこれらを準備すること。
> （していなければ演習そのものが実施できないだろう）

2.1. 参照する資料

演習の進め方や、Asignmentの受け取り方、必要なソフトウェア等については、Boxフォルダで配布した次の文書を参照のこ

コンパイラの講義用Boxフォルダの共有リンク
https://nihon-u.box.com/s/ea3bn1py3jcm1g5eoesw7og8iphwom3j

> 上記リンクは「リンクを知っている全員・表示およびダウンロード可能」となっている。

- 「コンパイラ」の講義資料
- asciidocプラグインの設定.pdf（Chromeブラウザ用、Visual Studio Code用）
- GitHub_Classroom_Assignmentの受け取り方_20220920.pdf
- gitコマンドの使い方.pdf
- github-git-cheat-sheet.pdf

> いちいちBoxのファイルを開いていては手間がかかるので、いったんデスクトップなどにダウンロードし、オフ
> インでも参照可能にしておくこと。

> もし、Boxドライブが閲覧できないようなら、共有申請すること。
> ただし、電子メールベースなので、応答には時間がかかることに注意。

Crafting Interpreters のウエブサイト
https://craftinginterpreters.com/

Crafting Interpreters のGitHubリポジトリ
https://github.com/munificent/craftinginterpreters/

2.2. ソフトウェアのインストール

下記のソフトウェアを使用する。未設定なものがあれば、インストールすること。

2.2.1. Visual Studio Code（VS Code）

インストールがまだなら
https://azure.microsoft.com/ja-jp/products/visual-studio-code

2.2.2. VS Codeのファイルエンコーディング設定

Windows版のVS Codeでは、テキストファイルのエンコーディング指定が MS932あるいはShift-JISとなっている場合がある。
習では、読む・書くともにUTF-8を使うことにするので、あらかじめエンコーディング設定を変更しておくこと。

VS Codeでエンコーディングを変更、自動判別するには
https://atmarkit.itmedia.co.jp/ait/articles/1806/01/news051.html

2.2.3. VS Code 機能拡張（プラグイン）の追加

- Java用の機能拡張をインストールする。
- Lox用の機能拡張をインストールする。
  - Lox用機能拡張も、Visual Studio Codeの機能拡張メニューから追加できる。詳しくは、下記Wikiを参照のこと。

    VS Code Lox Language
    https://marketplace.visualstudio.com/items?itemName=dberezin.lox-language

- インストールできたら、結果のわかるスナップを取得し、下記画像をそのスナップで置き換えなさい。
  - `images` ディレクトリにある `lox-language-plugin-installed.png` を上書き保存する。

> 後でプレビューを確認する。そのときに、下記のダミー画像が、自分の撮ったスナップに置き換わっていること
> 確認すること。

図 1. Lox用の機能拡張をインストールした

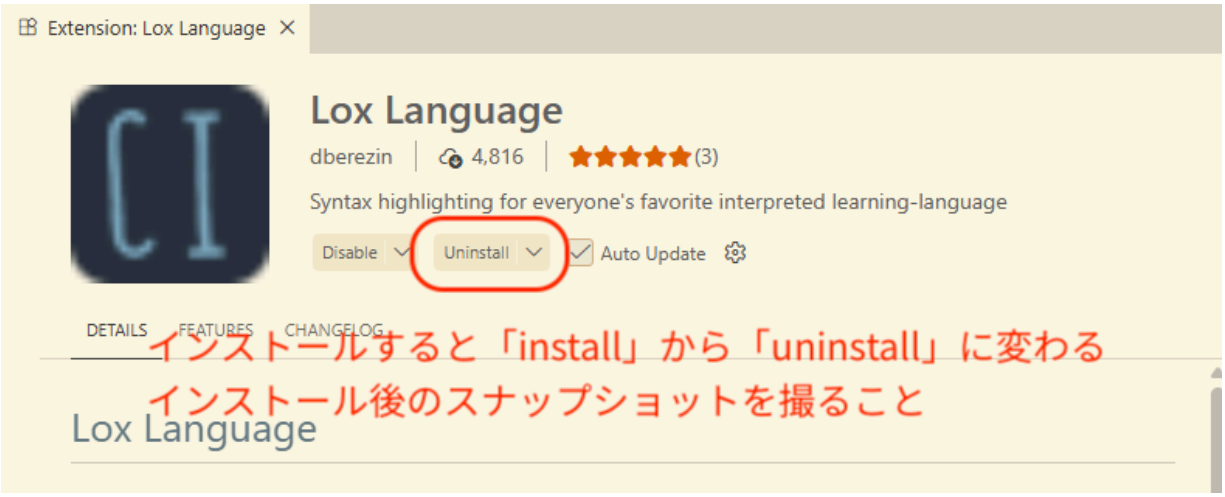

図 2. 実行した結果のスナップの例

スナップ（スクリーンショット）の撮り方は、次のページを参考にしなさい。

Windowsのスクリーンショットを撮る4つの方法
https://www.pc-koubou.jp/magazine/35994?srsltid=AfmBOorbVX7AYZLr4Vg7F5Zrp-4U3SbxxGypobX_REF7D-jCkLfK

2.2.4. Git

ここでの説明はWindows PCに、*「Git for Windows」*をインストールすることを想定している。

MacやWSL2上のGitを使う場合には、その方法に従うこと。

このAssingmentでは、この後、各所の操作説明等において、`Git for Windows` のインストーラを実行中に追加した `Git Bas`
うことを想定している。
もし、WSL2上のUbuntuや、MinGWまたはMSYS2のBashを使う、MacでZshを使うといった場合には、適宜読み替えて、自習環境用の `Git` を用意すること。

- 次のページを参照して、インストールする。
  - https://gitforwindows.org/
- わからないときは、次のページを参考にする。
  - https://www.curict.com/item/60/60bfe0e.html

2.2.5. Git Bashの動作確認

Gitコマンドが利用可能になっているか確認する。

1. `Git Bash` を実行する。
   - MSYS2やWSL2の人は、代わりに自分が演習で使うターミナルで実行すること。
2. `git --version` を実行する。
   - `git verion 2.49.0.windows.1` などバージョン番号が表示されればよい。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい。

図 3. Git Bashの動作を確認した

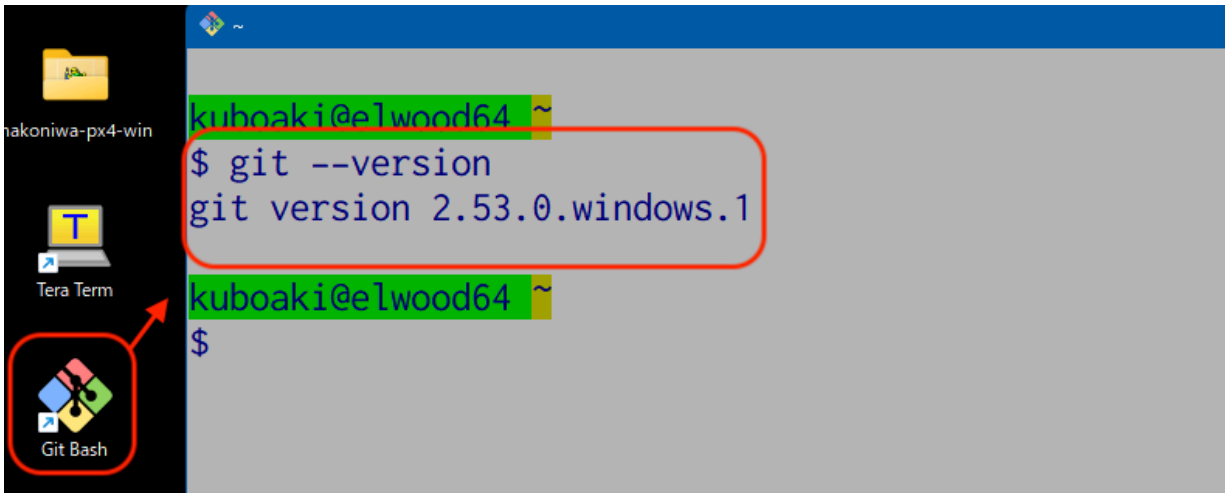

図 4. 実行した結果のスナップの例

2.2.6. Java開発環境（JDK）

Oracle JDKまたはOpenJDKをインストールする

- 次のページを参照して、インストールする。
  - https://www.oracle.com/java/technologies/downloads/
  - http://jdk.java.net/
- VS Codeを使う人は下記を参照してプラグインを導入するとよい。
  - https://note.com/liber_grp/n/n88f3f0a6fdf1

インストールできたか確認する。

> 古いJDKやJREが入っていると、演習のプログラムが **「コンパイルできない・動作しない」** といったことが起る。
> コントロールパネルの「プログラムの追加と削除」などで、古いものが残っていないか確認する。 もし残っていれば削除し、それから新しいものをインストールすること。

JDKがインストールできたか確認する

```
$ java --version  1
openjdk 21.0,6 2025-01-21 LTS  2
（略）
#javac --version  3
javac 21.0.6  4
```

1. `java` コマンドは、Javaのコンパイル済みプログラム（ `.class` や `.jar` ファイル）を指定して実行するインタプリタ。 `version` オプションでバージョンを表示している。
2. 自分がインストールしたJDKのバージョンと一致するか確認する。
3. `javac` コマンドは、Javaのソースコード(`.java` ファイル）のコンパイラ。 `--version` オプションでバージョンを表示する。
4. 自分がインストールしたJDKのバージョンと一致するか確認する。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい。

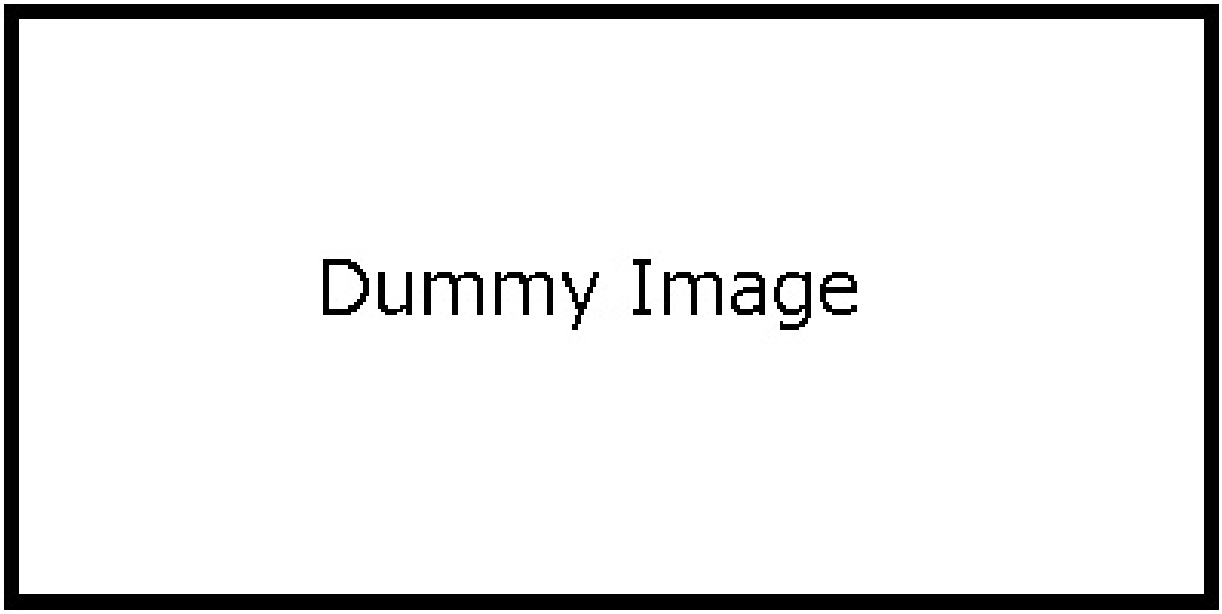

図 5. JDKのインストールを確認した

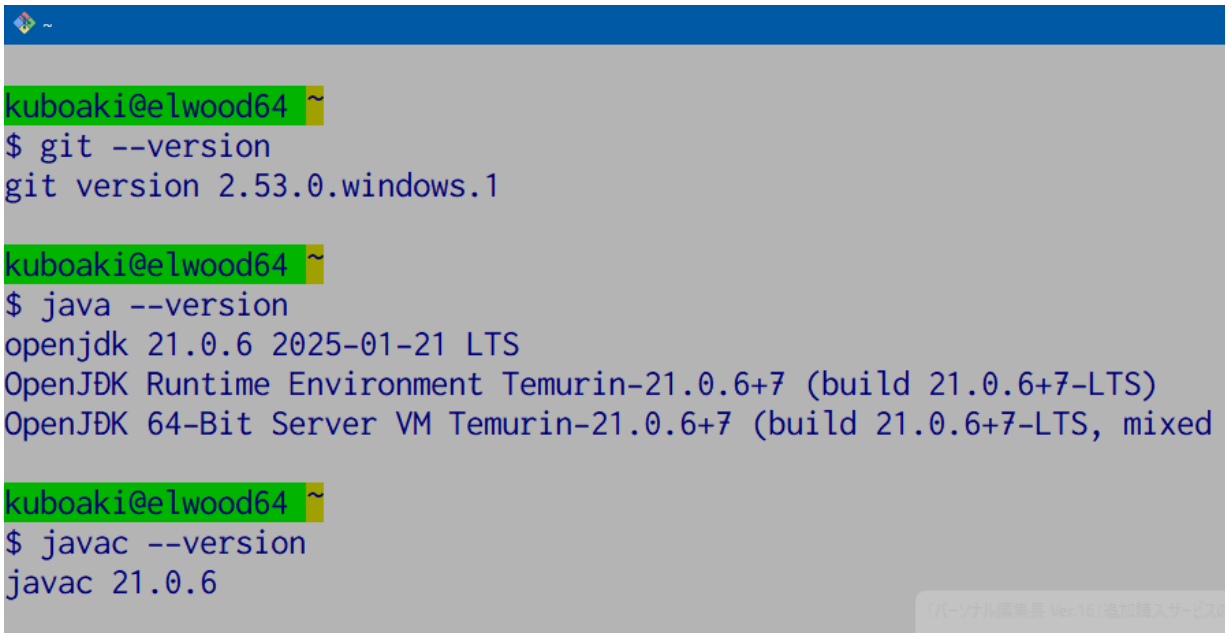

図 6. 実行した結果のスナップの例

2.2.7. Git Bashの設定

1. Windowsのタスクバーのスタートメニューの検索欄に「Git Bash」を入力して検索する。
2. ポップアップしたスタートアップメニューのアプリ欄に表示される「Git Bash」をクリックして起動する。
3. Git Bashのターミナルが表示される。
4. ターミナルの左上のアイコンをクリックし、ポップアップメニューを表示する。
5. ポップアップしたメニューから「Options」をクリックして、Optionsダイアログを表示する。
6. 左の「Looks」欄を選択し、右に表示された項目の「Theme」で「luminous」など、背景が明るいテーマから好みのも択する。
7. 左の「Text」欄を選択し、右に表示された項目の「Font」で「Select」ボタンをクリックし、日本語のフォントを選択ントサイズを少し大きめ（14ptから20pt程度）に変更する。
8. 右下の「Save」ボタンをクリックして設定を保存する。

> 下記実行例の $ はターミナルが表示するプロンプトを表す。 **自分の入力ではないので入力しないこと。**
>
> ターミナルのプロンプトは、実行する環境によって少しずつ異なる。 たとえば、ある環境では、実際に表示さるプロンプトは、
>
> `AaBbCc@LAPTOP-URPL5RT3 MINGW64 /y/nichidai/cit-complier-special (main)`
> `$ （ここで入力待ち）`
>
> のようになっていた。

Git Bashの動作を確認する

```
kuboaki@elwood64 ~
$ cd /c/Users/uboaki/Desktop/  1
kuboaki@elwood64 ~/Desktop
$ pwd  2
/c/Users/kuboaki/Desktop  3
kuboaki@elwood64 ~/Desktop
$ ls  4
 ImDiskTk-x64/        'Tera Term.lnk'*   desktop.ini
'Shin - Chrome.lnk'*   astah_sysml.lic   hakoniwa-px4-win/  5
```

1. `cd` はディレクトリの移動コマンド。この例では、 `c` がドライブ名、後に続くのが自分のデスクトップのディレクトリ。 `kuboaki` はログイン名。
2. `pwd` は、現在のディレクトリを表示するコマンド。
3. ディレクトリが移動できているか確認した。
4. `ls` コマンドで現在のディレクトリのファイルとディレクトリをリストする。
5. このディレクトリに含まれているファイルやディレクトが確認できた。名前の後ろに / がついているのはディレクトリ

2.3. GitHubアカウントの作成

GitHub classroomを使うには、日大のメールアカウントで作成したGitHubアカウントが必要。

もし、日大のメールアカウントを使用したGitHubアカウントを作成していないなら、講義資料(01-02回）や、下記ページを
て、個人用アカウント作成すること。

GitHub でのアカウントの作成
https://docs.github.com/ja/get-started/start-your-journey/creating-an-account-on-github

2.4. GitHubの Personal Access Token の作成

もし、Personal Access Token を作成していない（または使えなくなってしまっている）なら、ここで新たに作成する。

GitHubはベーシック認証（ユーザー名とパスワードによる認証）をやめている。 その代わりとして、コマンドラインからGi
アクセスするときにパスワード入力を求められたら、パスワードの代わりにあらかじめ作成しておいた Personal Access Tok
う。

下記手順を参考にして、 Personal Access Token を作成する。 詳しくは、講義資料(01-02回）を参考にするとよい。

1. PCでウェブブラウザを使ってGitHubにログインする。
2. ページ右上の自分のアカウントを示すアイコンをクリックしてポップアップメニューを表示する。
3. ポップアップしたメニューを下記のようにたどり、 `Personal access tokens (classic)` ページへ移動する。
   - `Settings ＞Developer settings＞Personal access tokens＞Tokens(classic)`
4. `Generate new Token` ボタンをクリックして、新しいトークンを作成する。
5. トークン名をつけるよう求められたら、 `for_pc_cli` など自分で用途がわかる名前をつけておく。
6. **アクセストークンはテキストファイルとしてデスクトップなどに保存しておく。**

> 作成したアクセストークンは、必ずすぐに分かる場所（デスクトップ等）に電子的に保管しておくこと。
> コマンドラインからGitHubを使うときに、パスワードの代わりに入力できるよう備えておく。 なくしたり、わ
> なくなったら、新しいトークンを作成してそれを使う。

2.5. ブラウザ用プレビューアの準備と確認

このファイル `README.adoc` のような Asciidoctor 形式のファイルをプレビューするため、プレビューアを用意する。

1. Chromeブラウザをインストールしていないなら、インストールする。
2. Chromeブラウザのウィンドウの右上、縦向きの ⋯ で示されるハンバーガーメニューを表示する。
3. ポップアップしたメニューを下記のようにたどり、「Chromeウェブストアにアクセス」ページへ移動する。
   - 機能拡張＞Chromeウェブストアにアクセス
4. 「機能拡張とテーマを検索」欄に「Asciidoctor」と入力して検索する。
5. 「Asciidoctor.js Live Preview」という機能拡張が見つかったら、「Chromeに追加」する。
   - ChromeブラウザのメニューアイコンのリストにAsciidoctorのアイコン（赤地の四角に`A`のアイコン）が追加され
6. 追加した機能拡張のアイコンを右クリックしてポップアップメニューを表示する。
7. ポップアップしたメニューから「オプション」をクリックして、オプション設定のページへ移動する。
8. ポップアップしているダイアログの「Asciidoctor options」の「Custom attributes」設定に下記を追加する。
   - `:sourcesdir: codes`
9. 追加できたら、ダイアログウインドウを閉じ、裏で表示されている機能拡張の設定ページに戻る。
10. 設定項目の中から「ファイルのURLへのアクセスを許可する」を探し、設定をONにする。
    - それでもchromeでローカルファイル（file:///で始まるファイル）にアクセスできない場合は、Chromeウェブスト
      「GoogleChrome でローカルファイルを開く」を追加する。
11. このファイル `README.adoc` をChromeブラウザで開いて、プレビューできることを確認する。
    - 追加した機能拡張のアイコンを左クリックするたびに、プレビュー表示と元のadocファイルの表示が切り替わる。

Assignmentを入手後、機能拡張の動作を確認する。

> もし、Chromeブラウザでadocファイルのプレビューが表示できない場合は、VS Codeのプラグインを使うか、
> レビューにこだわらずにテキストエディタでテキストファイルとして参照する。

2.6. VS Code用プレビューアの準備と確認

このファイル `README.adoc` のような Asciidoctor 形式のファイルをプレビューするため、プレビューアプラグインを導入す
だし、このプラグインは、AsciiDoc用であって、Asciidoctor 形式とは少し異なるため、表示が崩れてしまう場合があること
する。

1. VS Codeをインストールしていないなら、インストールする。
2. VS Codeの左下の歯車アイコンをクリックして、ポップアップメニューから「Extensions（機能拡張）」を選択する。
3. 左上の検索入力部分に「asciidoc」と入力して、機能拡張を検索する。
4. 検索結果から、 asciidoctor.org が署名している「AsciiDoc (AsciiDoc support for Visual Studio Code)」を選択する。
5. 編集ウィンドウ内の「AsciiDoc」のプロフィールから「Install」をクリックする。

Assignmentを入手後、機能拡張の動作を確認する。

2.7. 作業ディレクトリを作成する

ここでは、 Git Bash ターミナル（あるいは自分が用意した環境のターミナル）を使って作業することに注意。

演習用作業ディレクトリを作る

```
$ cd  1
$ mkdir compiler-work  2
$ cd compiler-work  3
```

1 引数なしの `cd` コマンドを使って、自分のホームディレクトリに移動する（引数なしで実行するとホームへ移動する）。

2 `mkdir` コマンドで `compiler-work` という名前のディレクトリを作る。

3 作成したディレクトリへ移動する。

既存の他のリポジトリのディレクトリの中に作業用ディレクトリを作らないこと。 そうするとリポジトリの入
ができてしまう。 リポジトリの入れ子があると、commitやpushでpermission エラーになることがある。

```
✅
├ lox01-AAA-BBB
├ cit-compiler-work  1

❌
├ lox01-AAA-BBB
        ├ cit-compiler-work  2
```

1 **OK**: 既存のリポジトリとは独立した場所に作成した。

2 **NG**: 既存のリポジトリである `lox01-AAA-BBB` の中に別のリポジトリを作ってしまった。

2.8. このAssignmentを受理（Accept）する

先述の「`GitHub_Classroom_Assignmentの受け取り方_20220920.pdf`」を参照して、このAssignmentを受理しなさい。

受理すると、自分のGitHubアカウントのホームディレクトリに「cit-compiler-special-アカウント名」のようなリポジトリた
る。

2.9. このAssignmentのリポジトリをCloneする

作成した作業ディレクトリにこのAssignmentのリポジトリをCloneしなさい。

このAsaignmentをCloneするためのURLを取得する

1. ウェブブラウザで、このAsaignmentのGitHubリポジトリページを開く。
2. ページ上部、やや右よりにある「<> Code」という緑のボタンをクリックして、ポップアップメニューを開く。
3. ポップアップしたメニューから「Local」タブ、「HTTPS」タブを選択し、「Clone using the web URL」の上の欄のURL
ーする。

演習用作業ディレクトリにAsaignmentをCloneする

```
$ cd （自分が用意した）/compiler-work  1
$ pwd  2
（自分が用意した）/compiler-work
$ git clone （取得したURL）  3
```

```
$ cd （リポジトリ名） 4
$ ls 5
```

1. 作業用ディレクトリ `compiler-work` へ移動する。
2. 作業用ディレクトリ `compiler-work` にいることを確認した。
3. `git clone` コマンドを使って、自分のAssignmentのリポジトリをCloneする。
4. cloneしたディレクトリへ移動する。
5. ディレクトリの内容が、cloneしたGitHubリポジトリに含まれているもの同じか確認する。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい。

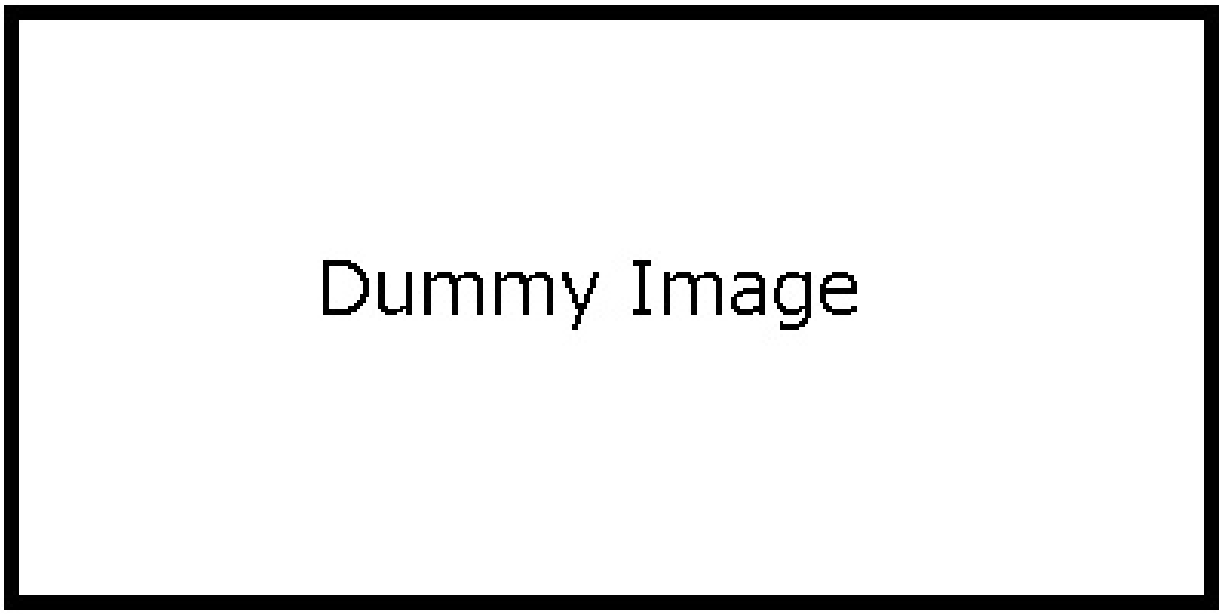

図 7. AssignmentをCloneしたディレクトリを確認した

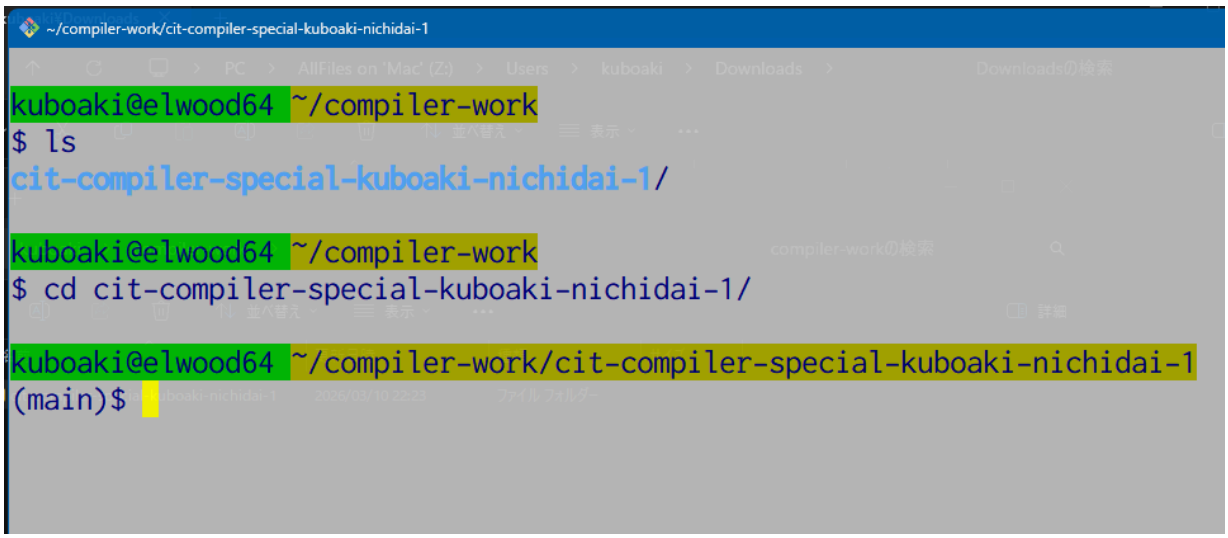

図 8. 実行した結果のスナップの例

2.9.1. Assignmentのリポジトリをターミナルから確認する

ターミナルから、Cloneしたリポジトリが参照できることを確認する。

リポジトリの確認（1）

```
（Assignmentのディレクトリにいるとする）
$ head -50 README.adoc 1

:linkcss:
:stylesdir: css
:stylesheet: mystyle.css
（中略）
== 演習の準備

[IMPORTANT]
--
基本的には講義中の演習を実施していたのであれば、これらの準部はすでに済んでいるはず。 +
もし、講義中に準備していなかったものがあれば、最初にこれらを準備すること。 +
```

（していなければ演習そのものが実施できないだろう） 2
--

1 head は、テキストファイルの先頭を表示するコマンド。 -50 オプションで、先頭から50行目までをから表示するようている。

2 Git Bashのターミナルで日本語が文字化けしていないことを確認した。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい。

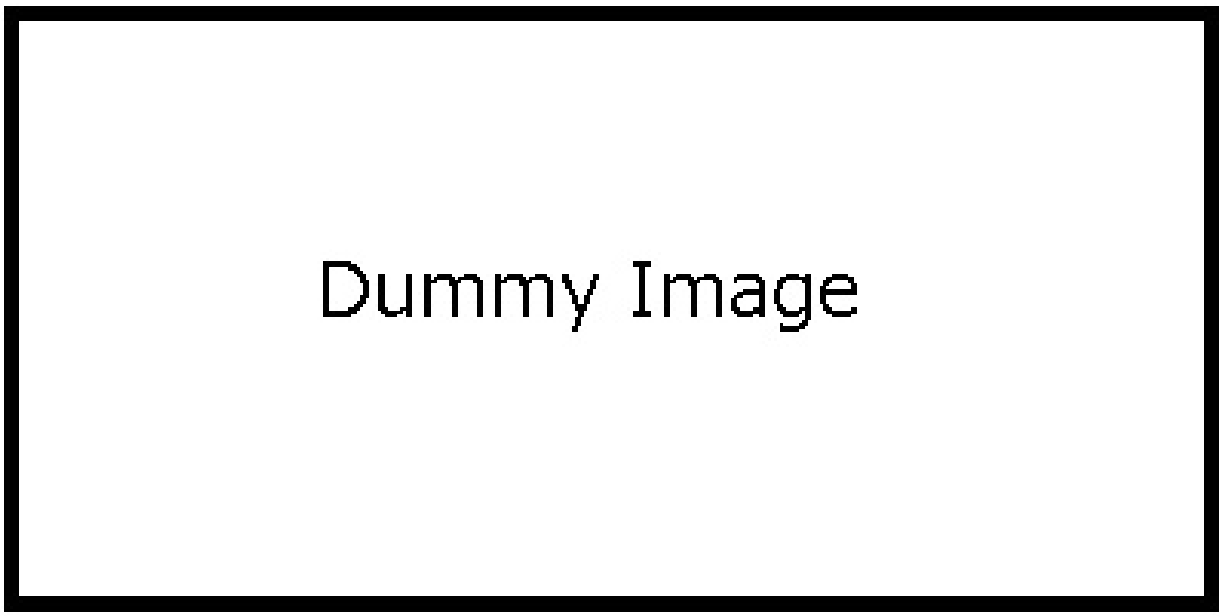

図 9. headコマンドでREADME.adocの中身を確認した

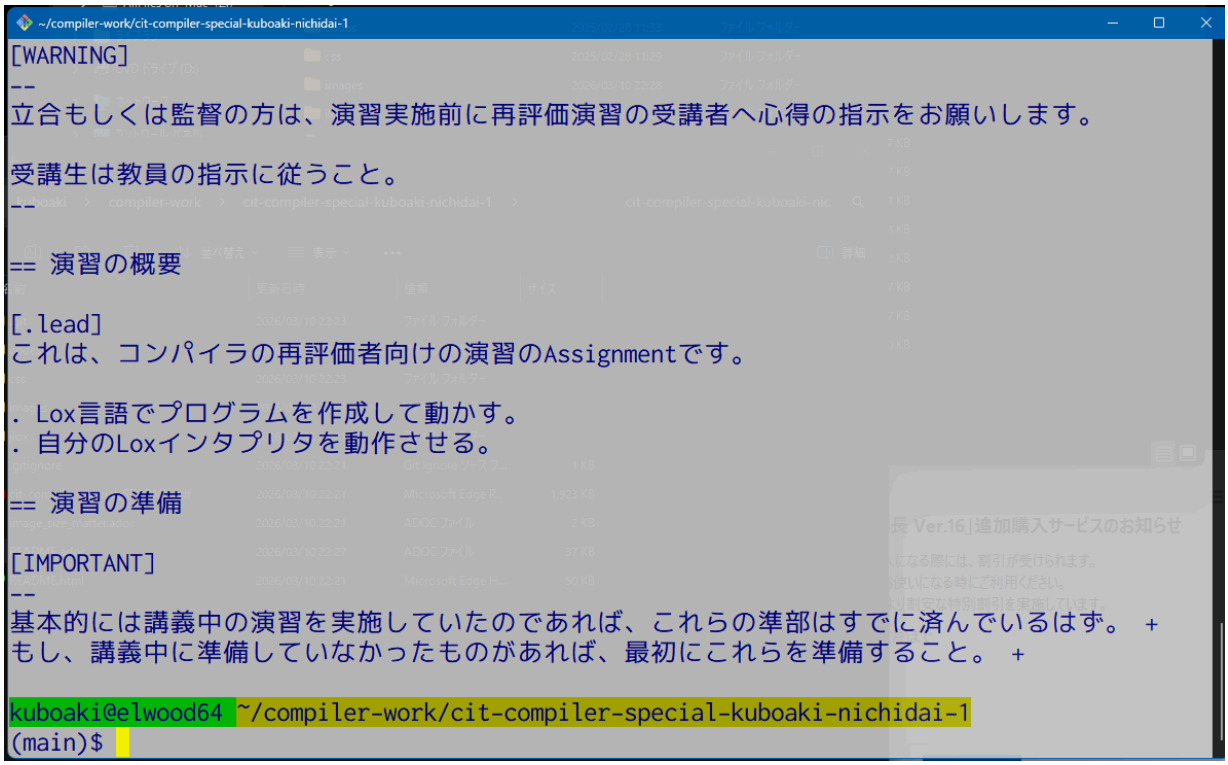

図 10. 実行した結果のスナップの例

2.9.2. Assignmentのリポジトリをブラウザから確認する

ブラウザで、Cloneしたリポジトリ一覧や、adocファイルが参照・プレビューできることを確認する。

リポジトリの確認（2）

1. chromeで、URLにこのREADMEファイルのあるディレクトリを指定し、ディレクトリリストが参照できるか確認する（ント名、ディレクトリ名は各自の環境に合わせて入力すること）。
   - file:///c:/Users/アカウント名/Assignmentのディレクトリ
2. 開けなかった場合は、Chromeウェブストアから「ローカルファイルリンク有効化」をChromeに追加する。
   - https://chromewebstore.google.com/detail/enable-local-file-links/nikfmfgobenbhmocjaaboihbeocacklc

3. ディレクトリリストが参照できるか確認する。
4. AsciiDocファイルがプレビューができるか確認する。
   - Chromeブラウザのアドレスバーの下段にあるプラグインのアイコン並びからAsciiDocプラグインのアイコンを探す（からないなら拡張機能メニューから探す）。
   - アイコンを右クリックしてポップアップメニューから「オプション」を選択する。
   - ポップアップする小さいウィンドウはそのまま閉じてよい。
   - 設定項目の中から「ファイルのURLへのアクセスを許可する」をONにする。
5. 設定後に再び `README.adoc` を開き、adocファイルがプレビューできることを確認する。

以後は、Asciidoctor形式のファイルをブラウザで開いていると、AsciiDocプラグインのアイコンをクリックするたびに、テ
形式とプレビューが切り替えられるようになる。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい（example02のようにプレビュー画面を撮る）。

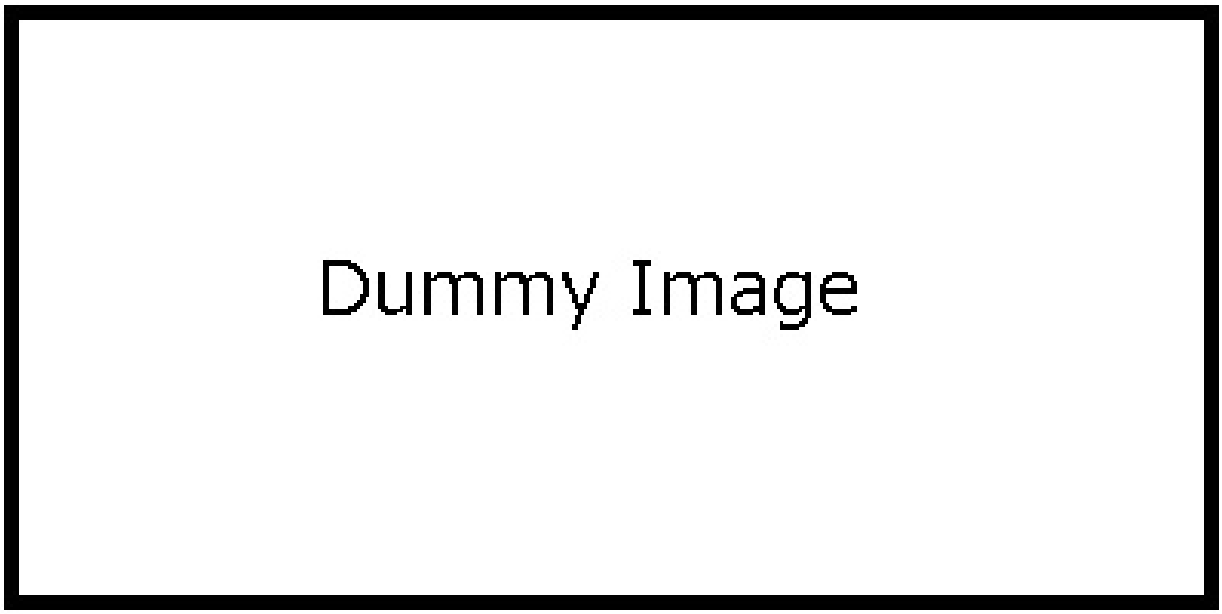

図 11. ChromeでREADME.adocをプレビューした

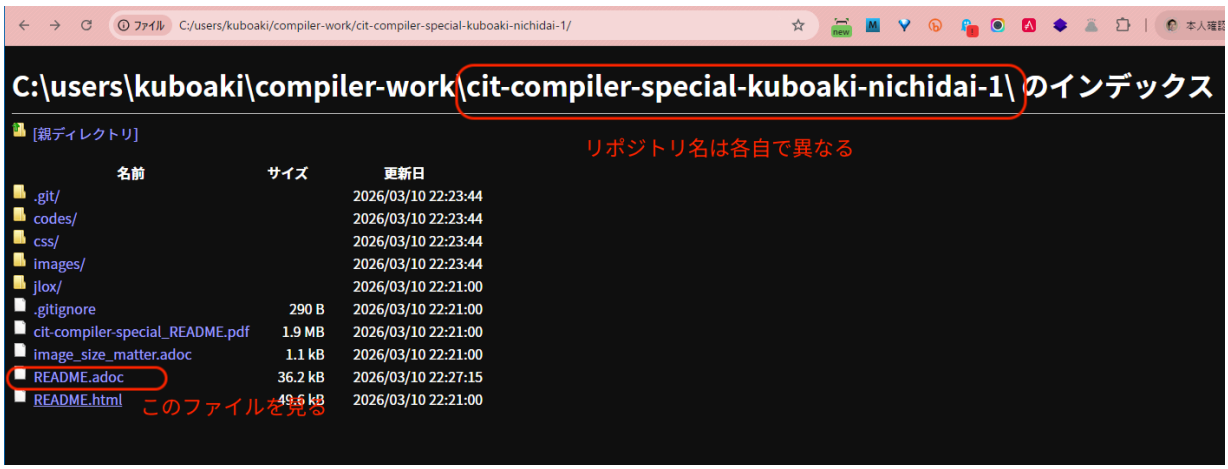

図 12. 実行した結果のスナップの例(1)

図 13. 実行した結果のスナップの例(2)

2.9.3. AssignmentのリポジトリをVS Codeから確認する

VS Codeで、Cloneしたリポジトリ一覧や、adocファイルが参照・プレビューできることを確認する。

リポジトリの確認（3）

1. VS Codeで、File＞Open Folderでファイルダイアログを開く。
2. Assignmentのディレクトリを選択し、「Select Folder」をクリックすると、フォルダーが左側にツリー展開される。
   - 初めて開くディレクトリの場合、「このフォルダーの作者を信頼するか？」というダイアログが表示されるので、「に
     頼します」を選択する。
3. ディレクトリのリストから、`README.adoc` を選択する。
4. ファイルが参照できたら、画面右から2段目にあるアイコンの並びから、「Open preview」アイコン（四角の真ん中に紙
   下に虫眼鏡のアイコン）をクリックする。
5. adocファイルがプレビューできていることを確認する。

ここでも、実行結果のスナップを撮って、下記の画像を置き換えなさい（example02のようにプレビュー画面を撮る）。

図 14. VS CodeでREADME.adocをプレビューした

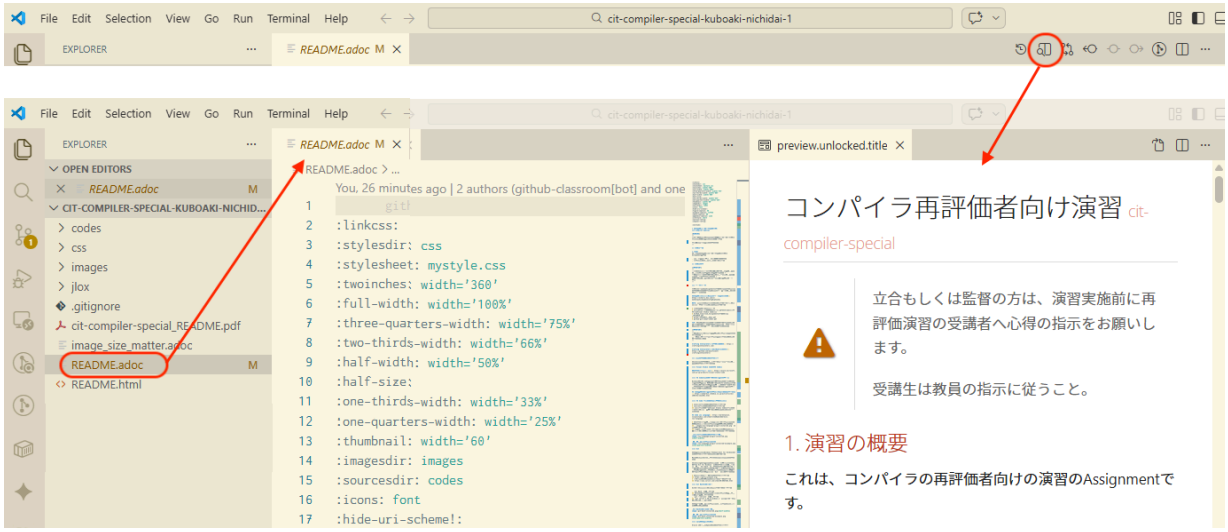

図 15. 実行した結果のスナップの例

3. 課題0

氏名等をこのファイルに記載する。

GitHub classroom からこのAssignmentをCloneしたら、このファイル（`REAMDE.adoc`）の下記部分を編集しなさい。

- 氏名: みょうじ なまえ
- 学籍番号: XXXXXXXX
- メールアドレス: XXXX@XXXXXXXX
- GitHubアカウント名: XXXXXXXX

このファイル（`README.adoc`）が編集できたら、コミットしてプッシュしなさい。

4. 課題1

作成済みのLoxインタプリタ（jLox）を使って、Loxのプログラムを動かしなさい。

cit-compiler-specialのディレクトリ構成

```
cit-compiler-special-xxxxxx  1
├──── README.adoc
├──── codes/
├──── css/
├──── image_size_matter.adoc
├──── images/
└──── jlox/  2
```

1 クローンしたリポジトリ名は、各自で異なる。

2 作成済みのjLoxはこのディレクトリに含まれている。

4.1. 作成済みのjLoxを動かす

1. このAssignmentの `jlox` ディレクトリへ移動する。
2. jloxプログラムをコンパイルする。
3. ソースディレクトリにclassファイルができているか確認する。
4. jloxプログラムを起動する。
5. サンプルプログラムを動かす。

実行例

```
$ pwd
/c/Users/アカウント名/compiler-work/cit-compiler-special-xxxxxx  1
$ cd jlox  2
$ ls
com/  3
$ javac com/craftinginterpreters/lox/Lox.java  4
$ ls com/craftinginterpreters/lox/*.class  5
```

```
 com/craftinginterpreters/lox/Environment.class
   （略）
 com/craftinginterpreters/lox/TokenType.class
$ java com.craftinginterpreters.lox.Lox  6
Welcom! Lox interpreter(jLox)
> print 1 +2;  7
3
> ^C  8
$
```

1. `pwd` コマンドで、現在のディレクトリがassignmentのディレクトリであることを確認した。
2. `cd` コマンドで、このAssignmentの `jlox` ディレクトリへ移動した。
3. `ls` コマンドで、ディレクトリのリストを表示した。
4. `javac` コマンドを使って、ソースコードをコンパイルした。
5. `ls` コマンドで、classファイルができているか確認した。
6. `java` コマンドを使って、 jLox を起動した。
7. 実行したいプログラムを入力し、結果が表示されるのを確認した。
8. `Ctrl + C` （CtrlキーとCキーを同時に押して）インタプリタを停止した。

上記実行例のように、jLoxのコンパイルからサンプルプログラムを実行するまでのターミナルのスナップを撮り、下記画像を
えなさい。

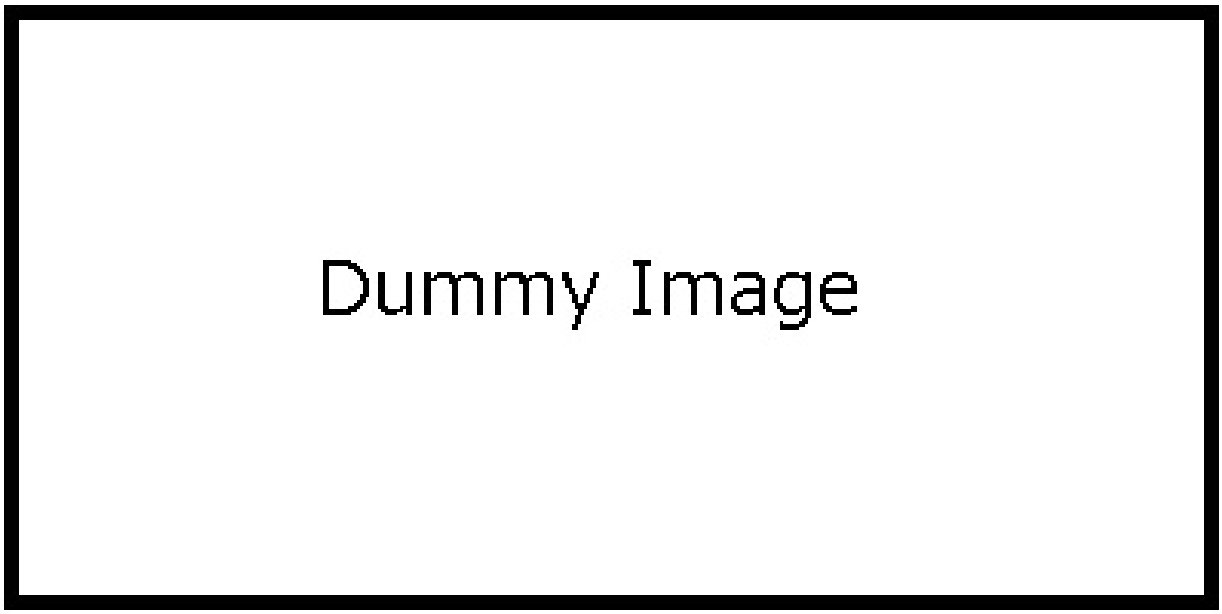

図 16. jLoxを動かした

4.2. 任意の場所でjLoxを動かす

他のディレクトリで jLox を利用したいときは、コマンドラインの `-cp` オプションを使って classpath を指定する。

たとえば、 このAssignmentの `codes` ディレクトリへ移動してから実行したいとするときは、次のようにする。

別のディレクトリでloxインタプリタを実行する

```
$ pwd
（どこかの）/cit-compiler-special/jlox
$ cd ../codes  1
$ ls  2
block_test.lox breakfast.lox sample01.lox sample02.lox
$ java -cp ../jlox com.craftinginterpreters.lox.Lox sample02.lox  3
60
false
CCCccc
true
0.5
```

1. `cd` コマンドを使って、 `codes` ディレクトリへ移動した。
2. このディレクトリに `sample02.lox` があることを確認した。
3. `java` コマンドに `-cp` オプションで jlox の場所を指定し、 `sample02.lox` を実行した。

実行した様子のスナップで下記画像を置き換えなさい。

上記実行例のように、jLoxのコンパイルからサンプルプログラムを実行するまでのターミナルのスナップを撮り、下記画像を
えなさい。

図 17. jLoxを使って別のディレクトリにあるプログラムを動かした

5. 課題2

5.1. Loxプログラムを作成して動かす

次のLoxプログラム `breakfast.lox` を `codes` ディレクトリに作成しなさい。

breakfast.lox U ×

codes > breakfast.lox > ...

```
class Breakfast {
  init(meat, bread) {
    this.meat = meat;
    this.bread = bread;
  }
  cook() {
    return "Cooking now ... " + this.meat + " and " + this.bread + ".";
  }
  serve(who) {
    return "Enjoy your breakfast(" + this.meat + ", " + this.bread + "), " + who + ".";
  }
}

var breakfast = Breakfast("sausage", "toast");
print breakfast.cook();
print breakfast.serve("Gentleman");
print clock();
```

図 18. 作成するLoxプログラム（`breakfast.lox`）

作成できたら、 4.2 に示した方法で実行しなさい。

`breakfast.lox` を実行する

```
$ pwd
（どこかの）/cit-compiler-special/coedes
$ ls
breakfast.lox breakfast02.bnf  personal infomation.csv
sample01.lox  sample02.lox  tiny_lang.bnf
$ java -cp ../jlox com.craftinginterpreters.lox.Lox breakfast.lox
（実行結果）
```

1. `pwd` コマンドを使って、 `codes` ディレクトリにいることを確認した。
2. このディレクトリに `breakfast.lox` があることを確認した。
3. `java` コマンドに `-cp` オプションで jlox の場所を指定し、 `breakfast..lox` を実行した。

`breakfast.lox` をターミナル上で実行している様子のスクリーンショットを撮りなさい。

図 19. `breakfast.lox` を実行した結果

5.2. 作業を保存する

このAsainnmenntのリポジトリをコミットして、プッシュしなさい。

```
$ pwd  1
（どこかの）/cit-compiler-special/
$ git add .  2
$ git status  1
$ git commit -m "update for Ex.1"  3
$ git status  4
$ git push  5
```

1. `git status` コマンドを使って、現在のディレクトリを確認。もし、リポジトリの外のディレクトリにいたなら、ここに
   してくる。
2. `git add` コマンドを使って、更新対象のファイルをすべてステージに追加する（コミットの対象として指定する）。
3. `git commit` コマンドを使って、更新状態を保存する。 `-m` の後ろはコミットメッセージ。
4. コミットできたか確認した。
5. `git push` コマンドで、GitHub上のリポジトリへ更新分を反映する。

6. 課題3

Javaを使って、講義で実施した範囲をカバーするLoxのインタプリタを作りなさい。

6.1. 開発用ディレクトリを作成する

開発用ディレクトリを作成する

```
$ pwd  1
（このAssignmentのディレクトリ）
$ cd codes  2
$ mkdir com  3
$ mkdir com/craftinginterpreters
$ mkdir com/craftinginterpreters/lox
$ touch com/craftinginterpreters/lox/Lox.java  4
```

1. 現在のディレクトリがAssignmentのディレクトリであることを確認した。
2. このAssignmentのリポジトリの `codes` ディレクトリへ移動する。

3 `mkdir` コマンドを使って、 `codes` ディレクトリに下記のディレクトリ階層を作成する。

4 `touch` コマンドを使って、 `com/craftinginterpreters/lox/` ディレクトリに `Lox.java` ファイルを作った。

6.2. テキストの4章を作成する

1. VS Codeなどのテキストエディタで、4章までのLoxのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルしなさい。

実行例

```
(codesディレクトリにいる)
$ javac com/craftinginterpreters/lox/Lox.java  1
$ ls com/craftinginterpreters/lox/*.class  2
 com/craftinginterpreters/lox/Environment.class
   (略)
 com/craftinginterpreters/lox/TokenType.class
```

1 `javac` コマンドを使って、作成したプログラムをコンパイルした。

2 `ls` コマンドで、classファイルができているか確認した。

3. 次のLoxプログラム `doughnut.lox` を `codes` ディレクトリに作成しなさい。

doughnut.lox U ×

codes > doughnut.lox > ...

```
// doughnut.lox
class Doughnut {
    cook() {
        print "Fry until golden brown.";
    }
}
class BostonCream < Doughnut {
    cook() {
        super.cook();
        print "Pipe fill of custrard and coast with chcolate.";
    }
}

BostonCream().cook();
```

図 20. 作成するLoxプログラム（ `doughnut.lox` ）

4. 下記手順を参考に、作成した `doughnut.lox` を実行しなさい。

実行例

```
(codesディレクトリにいる)
$ java com.craftinginterpreters.lox.Lox doughnut.lox  1
CLASS class null
IDENTIFIER Doughnut null
LEFT_BRACE { null
   (略)
PRINT print null
STRING "Pipe fill of custrard and coast with chcolate." Pipe fill of custrard and coast with chcolate.
SEMICOLON ; null
RIGHT_BRACE } null
   (略)
RIGHT_PAREN ) null
SEMICOLON ; null
EOF  null
> ^C  2
```

1 `java` コマンドを使って、 jLox を起動した。実行プログラムとして `doughnut.lox` を指定した。

2 `Ctrl + C` （CtrlキーとCキーを同時に押して）インタプリタを停止した。

ターミナルで、4章までのコードを使って `doughnut.lox` を実行している様子のスクリーンショットを撮りなさい。

図 21. `doughnut.lox` を実行した結果

> 4章が終わったら、下記手順を参考に、コミットしてプッシュしなさい。

変更をコミット（commit）する

```
$ git status  1
$ git add .  2
$ git status  3
$ git commit -m "4章まで作成した"  4
$ git push  5
Username for 'https...（リポジトリのURL）': （GitHubアカウント名）  6
Password for 'https:...（リポジトリのURL）': （Personal access token）  7
```

1. 変更点を確認した。
2. 追加・更新したファイルをコミット対象に追加した。
3. もう一度Statusで追加できていることを確認した。
4. コミットした。
5. GitHub側のリポジトリへプッシュする。
6. ユーザー名を求められたら、 GitHubアカウント名を入力する。
7. パスワードを求められたら、パスワードの代わりにPersonal access tokenを入力する。

7. 課題4

7.1. テキストの5章を作成する

1. Visual Studio Codeなどのテキストエディタで、5章までのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルし、実行しなさい。

実行できるか確認する

```
$ javac com/craftinginterpreters/lox/AstPrinter.java  1
$ java com.craftinginterpreters.lox.AstPrinter  2
(* (- 123) (group 45.67))
```

1. `AstPrinter.java` をコンパイルした。
2. `AstPrinter` を実行した。

図 22. doughnut.lox を実行した結果

ターミナルで、5章までのコードを使って 上記サンプルを実行している様子のスクリーンショットを撮りなさい。

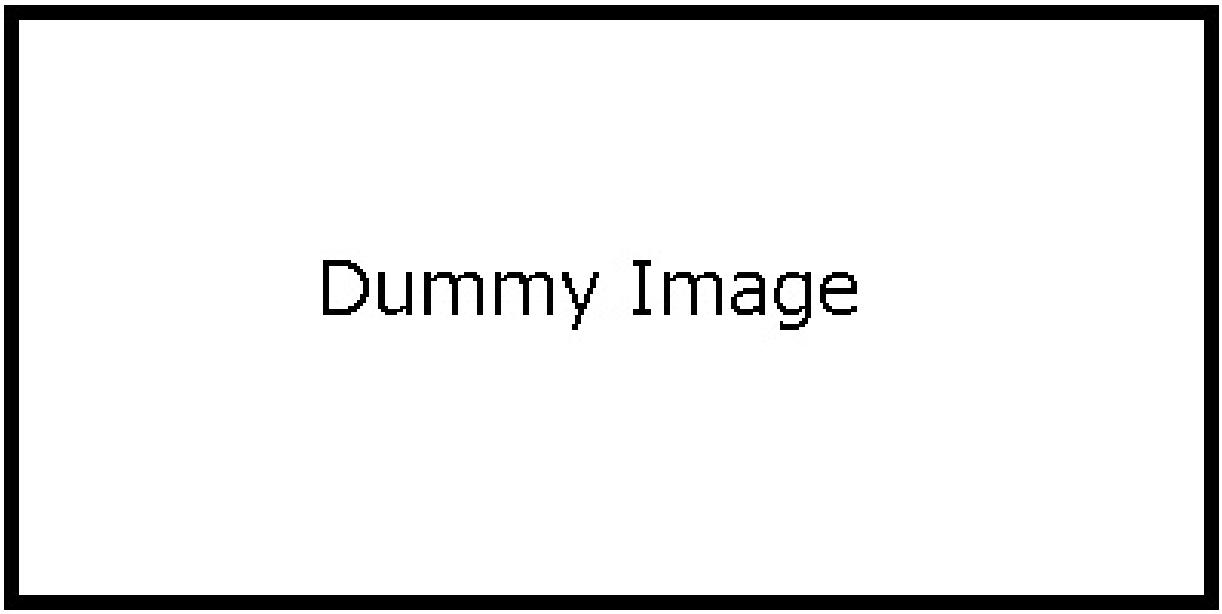

図 23. 5章の実行結果

5章が終わったら、コミットしてプッシュしなさい。

7.2. テキストの6章を作成する

1. Visual Studio Codeなどのテキストエディタで、6章までのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルし、実行しなさい。

実行できるか確認する

```
$ javac com/craftinginterpreters/lox/Lox.java  1
$ java com.craftinginterpreters.lox.Lox  2
> 4 * ( 5 + 3 )
(* 4.0 (group (+ 5.0 3.0)))  3
```

1. Lox.java をコンパイルした。
2. Lox を実行した。
3. Loxが、5章のAstPrinterと同じように動作するようになった。

ターミナルで、6章までのコードを使って 上記サンプルを実行している様子のスクリーンショットを撮りなさい。

図 24. 6章の実行結果

6章が終わったら、コミットしてプッシュしなさい。

8. 課題5

8.1. テキストの7章を作成する

1. Visual Studio Codeなどのテキストエディタで、7章までのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルし、実行しなさい。

実行できるか確認する

```
$ javac com/craftinginterpreters/lox/Lox.java  1
$ java com.craftinginterpreters.lox.Lox  2
> 1 + 1  3
2
> 2 * ( 3 + 4 )
14
>
```

1 `Lox.java` をコンパイルした。

2 `Lox` を実行した。

3 Loxが、式を与えると計算結果を返すようになった。

ターミナルで、7章までのコードを使って 上記サンプルを実行している様子のスクリーンショットを撮りなさい。

図 25. 7章の実行結果

7章が終わったら、コミットしてプッシュしなさい。

8.2. テキストの8章を作成する

1. Visual Studio Codeなどのテキストエディタで、8章までのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルし、実行しなさい。

実行できるか確認する

```
$ javac com/craftinginterpreters/lox/Lox.java  1
$ java com.craftinginterpreters.lox.Lox block_test.lox  2
inner a  3
outer b
global c
outer a
outer b
global c
global a
global b
global c
```

1 `Lox.java` をコンパイルした。

2 `Lox` を使って `block_test.lox` を実行した。

3 Loxが、ブロックと変数のスコープを解釈できるようになった。

ターミナルで、8章までのコードを使って 上記サンプルを実行している様子のスクリーンショットを撮りなさい。

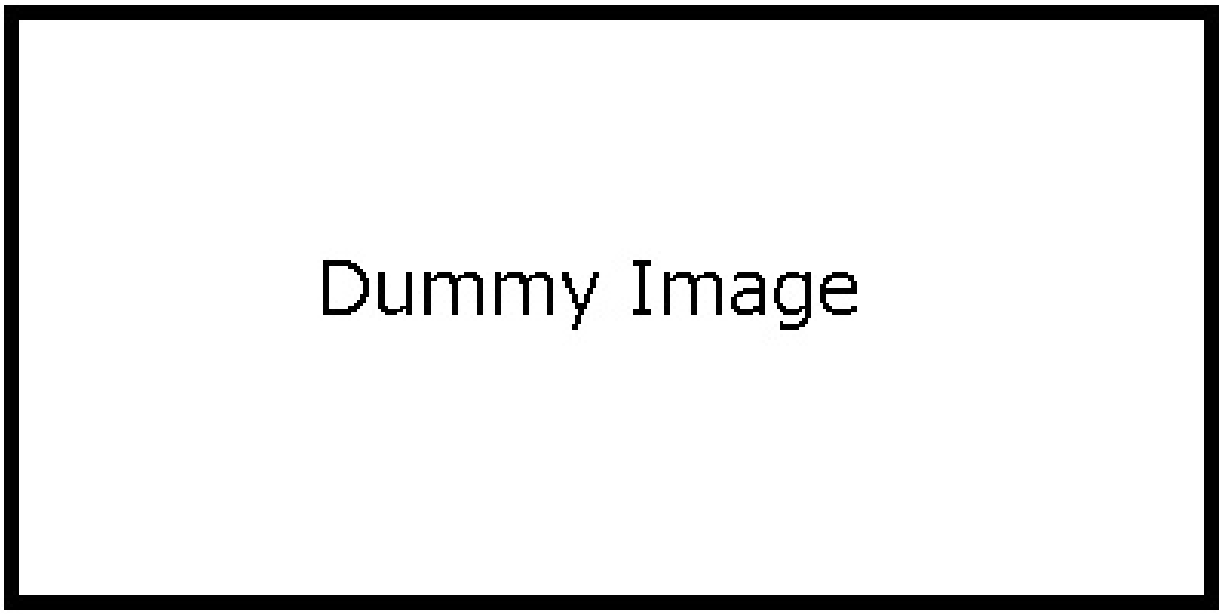

図 26. 8章の実行結果

8章が終わったら、コミットしてプッシュしなさい。

8.3. テキストの9章を作成する

1. Visual Studio Codeなどのテキストエディタで、9章までのコードを打ち込み、保存しなさい。
2. 次の実行例に従ってプログラムをコンパイルし、実行しなさい。

実行できるか確認する

```
$ javac com/craftinginterpreters/lox/Lox.java  1
$ java com.craftinginterpreters.lox.Lox sample02.lox  2
60  3
false
CCCccc
true
0.5
```

1. `Lox.java` をコンパイルした。
2. `Lox` を使って `block_test.lox` を実行した。
3. Loxが、ブロックと変数のスコープを解釈できるようになった。

ターミナルで、9章までのコードを使って 上記サンプルを実行している様子のスクリーンショットを撮りなさい。

chap09 result
図 27. 9章の実行結果

9章が終わったら、コミットしてプッシュしなさい。

9. 課題6

本講義の演習に使用したテキストのAmazonの商品ページのカスタマーレビュー欄に、このテキストを使って演習したことにカスタマーレビューを投稿しなさい。

Amazonでは、Amazonで商品を購入していない場合（他社で購入など）についても、その商品のカスタマーレビューを投稿できるようになっている。 ただし、Amazonで購入した場合とは、掲載する数や掲載までの時間に違ある。

「インタプリタの作り方 ー言語設計／開発の基本と2つの方式による実装ー」
https://www.amazon.co.jp/dp/4295017876?ref_=ppx_hzsearch_conn_dt_b_fed_asin_title_2

1. Amazonのウェブサイトにアクセスする。
   - https://www.amazon.co.jp/
2. Amazonにアカウントを持っていないなら、作成する。
3. ログインする。
4. 上記書籍のページを開く。
5. ページ後半の「カスタマーレビュー」の中の「カスタマーレビューを書く」をクリックして、カスタマーレビュージを開く。
6. ☆をつけ、レビューを書く。適宜タイトルもつけること。
7. 投稿は、すぐに掲載されない。Amazonのチェックが済むと掲載される。
8. 書評が掲載されたら、それを本課題の成果とする。

掲載された書評のスナップを撮り、下記画像を置き換えなさい。

amazon review result
図 28. Amazonに掲載された書評

10. 再評価演習の提出

このAssignmentの成果（READMEの更新、codesディレクトリの中身など）を確認して、最終的なコミットとプッシュを実行
と。

このAssignmentについて、最終的な commit、pushの結果をもって再評価演習を提出したとみなす。

Last updated 2026-03-11 00:52:44 +0900